Programming Fundamentals (CT-175)

Complex Computing Activity (CCA) – Project Report

Hospital Management System

Submitted By

S. No.	Student Name	Roll Number	Section
1	Muhammad Yousuf Jamil	CT-26292	F
2	Syed Omer Abdul Rehman	CT-26291	F
3	Muhammad Owais	CT-26278	F
4	Asad Abbas	CT-26280	F

Submitted To

Instructor: Ms. Maham Ashraf

Course: Programming Fundamentals (CT-175)

Semester: Fall 2026

Submission Date: 11 / 11 / 2026

GitHub Repository: https://github.com/myj-world/PF_CCA_Project_GroupNo4

1. Introduction

In modern healthcare administration, efficient programs are essential for maintaining quality patient care, managing resource distribution, and preventing administrative bottlenecks. The Hospital Management System (HMS) is a modules-based and console-based application developed in C to manage a small-scale medical facility. Designed as part of the Programming Fundamentals Complex Computing Problem/Activity (CCP/CCA), the system provides a centralized platform for handling patient registration, dynamic queue management, doctor scheduling, multi-variable medical billing, and persistent record storage. By dividing the software into administrative, reception, doctor, and reporting portals, the system makes sure it is simple for everyone to use.

The project addresses several constraints by combining static data structures and dynamic memory allocation. While doctor profiles, daily appointments, and financial records are maintained using specialized structures and fixed-capacity arrays, patient records dynamically scale using runtime allocation (malloc and realloc) to handle changing administrative loads safely. To resolve scheduling conflicts, the system maps doctor availability on a two-dimensional grid matrix showing the hourly time slots (09:00 AM to 04:00 PM), hence resolving the double-booking issue. Furthermore, a priority queue sorting algorithm ensures that emergency medical cases jump the queue while maintaining order of arrival for non-emergency patients. Persistent binary file operations (hospital_data.txt) ensure data remains saved across sessions, along with binary corruption checks on program start.

2. Objectives

To develop a portal-based hospital management system in C using switch to allow user to select options

- To implement dynamic memory allocation (malloc and realloc) for growing patient records, alongside a 2D matrix (schedule[MAX_DOC][MAX_SLOT]) to prevent doctor double-booking
- To design solutions for emergency patients, using custom priority sorting (sortQueue) to prioritize critical patients and recursive functions for using binary search to search for ID, and recursion for revenue calculations.
- To ensure persistent data storage and system reliability by saving/loading records to a binary file (hospital_data.txt) with startup corruption checks.
- To validate all user inputs (takeValidInt, takeStringInput) so that invalid choices, out-of-bounds parameters, and input buffer glitches are prevented.

3. Features

No.	Feature	Description
1	Patient Management	Registers new patients with dynamic memory allocation, updates details, and maintains insurance class information.
2	Doctor Management	Adds doctors with specializations and consulting fees, avoiding duplicate doctor IDs in the system.
3	Appointment Scheduling	Books time slots for patients, validates specialization availability, and assigns room/treatment types.
4	Priority Queue for emergency	Automatically sorts patient queues based on emergency status and booking order.
5	Billing and Insurance calculation	Calculates total charges (consultation, room, treatment) and applies insurance discounts according to coverage tiers.
6	Payment Processing	Handles unpaid bills, tracks payment methods, calculates change due, and marks bills as paid.
7	Reports and Revenue Calculation	Displays real-time schedules, active patient queues, and computes total hospital revenue recursively.
8	Data Persistance	Saves and loads all patient, doctor, appointment, bill, and schedule data to binary text files.

4. Programming Concepts Used

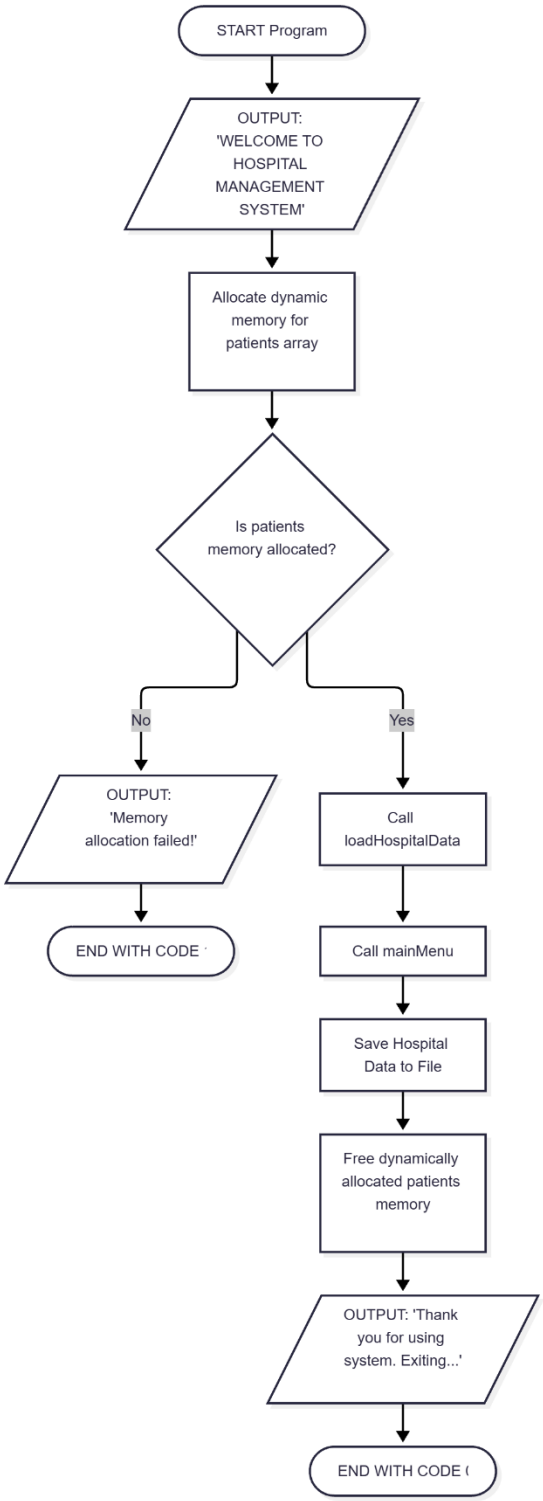
Concept	Where it is used in the project	Why it is used
Variables and data types	Used across all modules (int, float, char, bool) like pCount, dCount, finalAmount, and insured	To hold primitive data values, record counters, fees, status flags, and dynamic memory capacities.
Conditional statements (if / switch)	mainMenu(), receptionMenu(), calculateBill(), applyInsurance(), and input validation helpers	To route menu actions, evaluate user input choices, and compute tiered insurance discounts
Loops	mainMenu(), takeValidInt(), sortQueue(), sortPatients(), and search functions	To iterate through arrays, handle menu repetition, validate input ranges, and execute sorting algorithms
Functions	System is modularized into menu controllers, helper utilities, billing functions, and search algorithms	To break the program into reusable, manageable sub-routines and maintain clean code structure.
Arrays / Strings	doctors[100], appts[5000], bills[5000], schedule[100][8], and string functions (strcmp, strcpy)	To store collections of hospital records, doctor schedules matrix, and patient text details
Structures	Custom data types: Patient, Doctor, Appointment, Bill, and SlotTime.	To group related attributes of

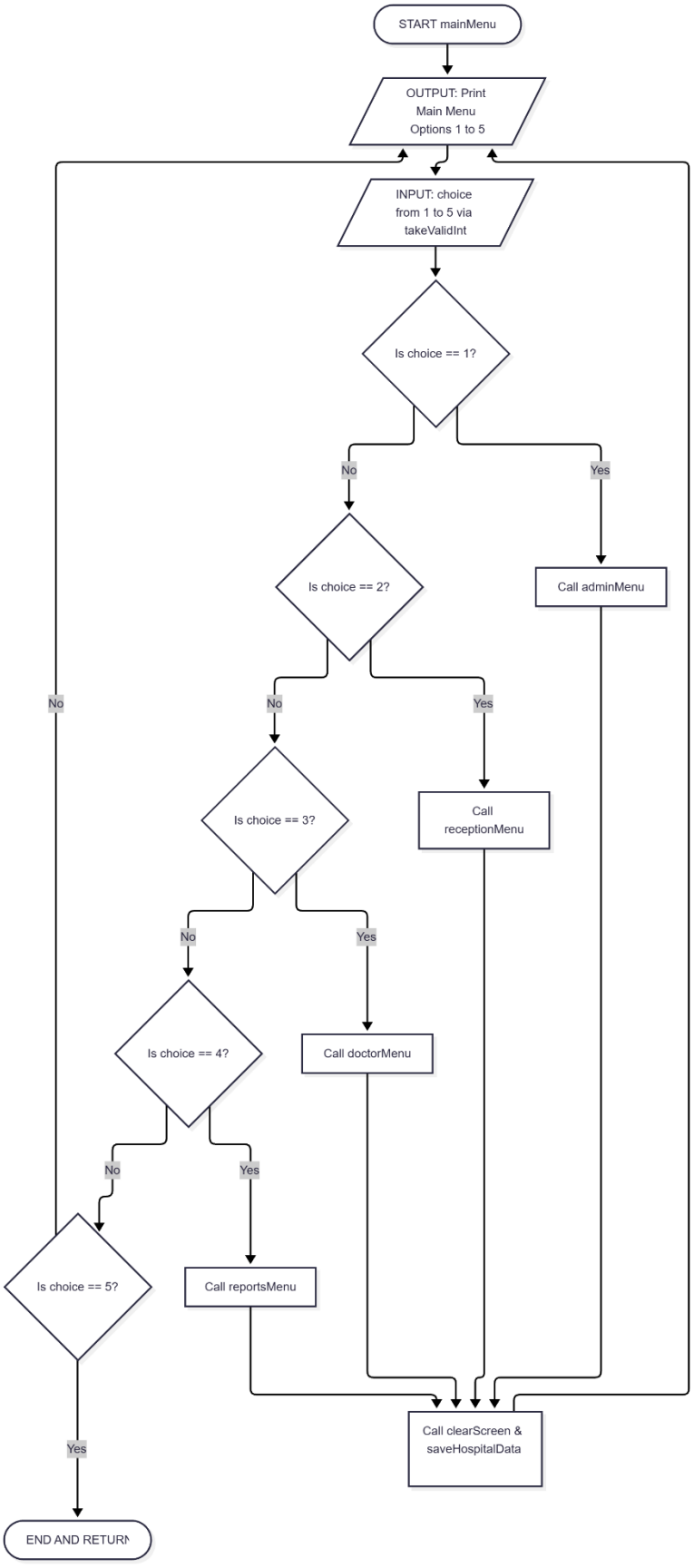
Concept	Where it is used in the project	Why it is used
		hospital entities together into single objects.
Pointers / Dynamic memory	patients dynamic array using malloc() and realloc() in main() and registerPatient()	To dynamically expand patient storage capacity at runtime as new patients are registered.
File handling	Binary text file functions fopen(), fwrite(), fread(), and fclose() in saveHospitalData() and loadHospitalData()	To save application state to hospital_data.txt so data persists across program executions.
Recursion	recursiveBinarySearchID() for patient lookup and recursiveTotalRevenue() for financial calculations.	To perform logarithmic binary searching on sorted IDs and calculate total revenue recursively.

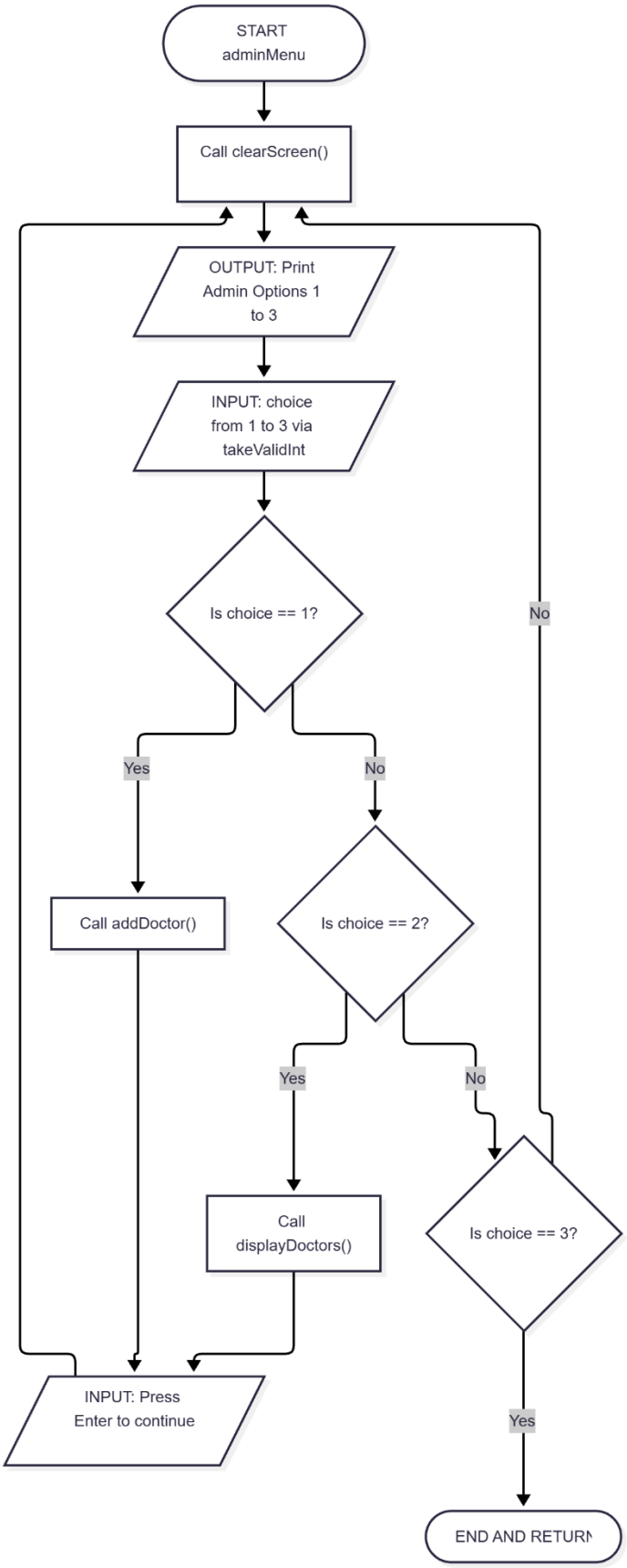
5. Algorithm

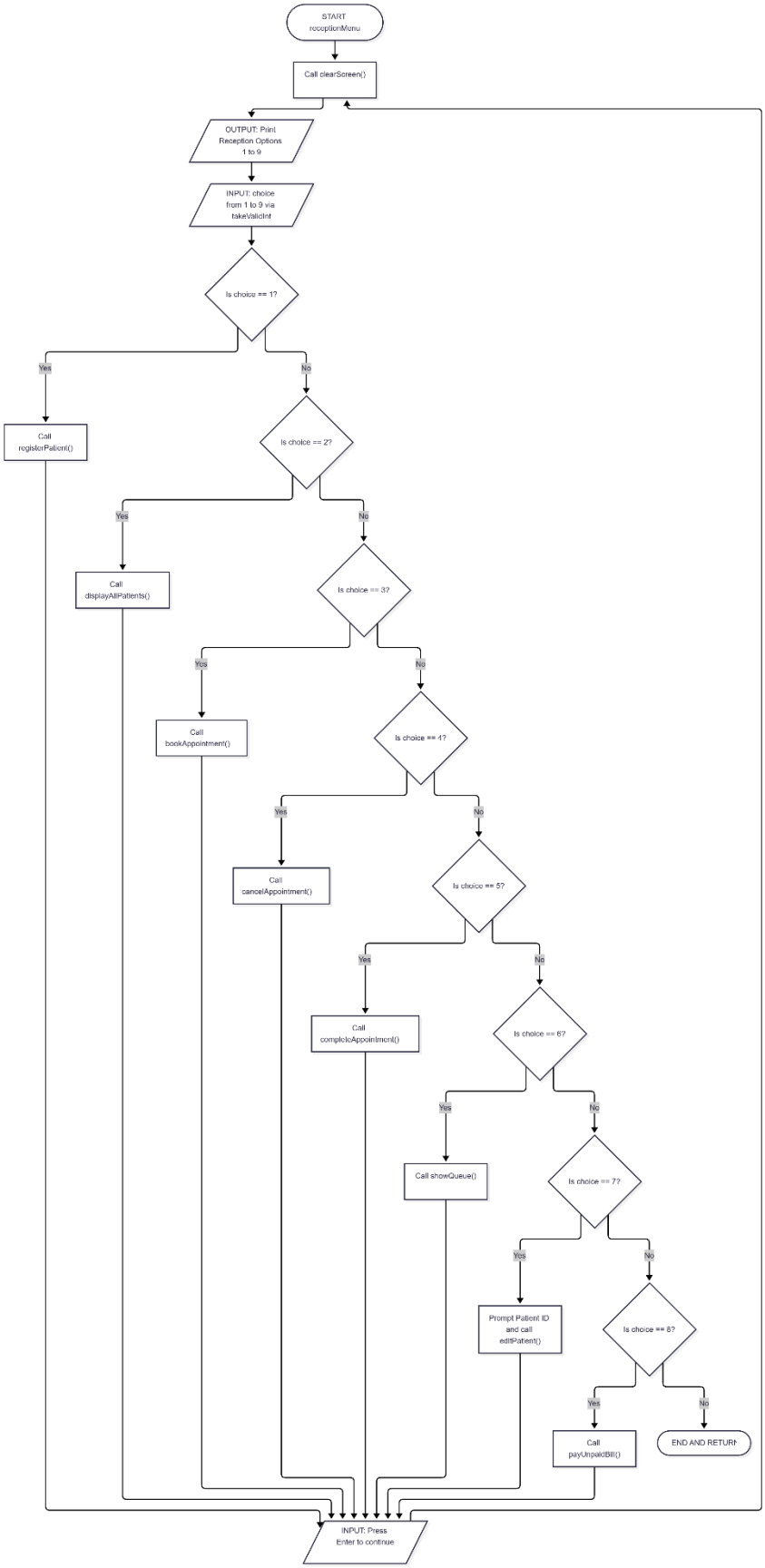
1. Start the program and allocate dynamic memory for the patients array using malloc().
2. Attempt to load previous hospital data (patients, doctors, appointments, bills, schedule grid) from the binary file hospital_data.txt. If the file does not exist or is corrupted, initialize record counters (pCount, dCount, aCount, bCount) to 0 and start fresh.
3. Display the main menu with 5 options: Admin Portal, Reception Portal, Doctor Portal, Hospital Reports & Schedule, and Exit/Save.
4. Read user input choice and validate it using takeValidInt(1, 5). If the input is outside the valid range, prompt an error message and repeatedly request input until a valid choice is entered.
5. Execute selected portal based on user choice:
    a. Admin Portal: Add new doctors (verifying duplicate doctor IDs) or view all doctors.
    b. Reception Portal: Register new patients (expanding dynamic memory via realloc() if capacity is reached), view patients, book appointments (checking free doctor slots), cancel/complete appointments, edit patient details, view triage queues, and pay unpaid bills.
    c. Doctor Portal: View the complete doctor daily schedule matrix.
    d. Reports Menu: View patient/doctor lists, view schedules, view triage queues, and calculate total hospital revenue recursively using recursiveTotalRevenue().
6. Automatically sort patient queues on new bookings based on emergency status (give priority) and appointment ID using sortQueue().
7. Calculate bills and apply insurance upon appointment completion based on room type, treatment type, and insurance class tier limits via applyInsurance().
8. Repeat the main menu loop until the user explicitly selects option 5 (Exit / Save).
9. Save all updated hospital arrays and counters to hospital_data.txt using saveHospitalData().
10. Free dynamically allocated patient memory using free() and terminate the program cleanly.

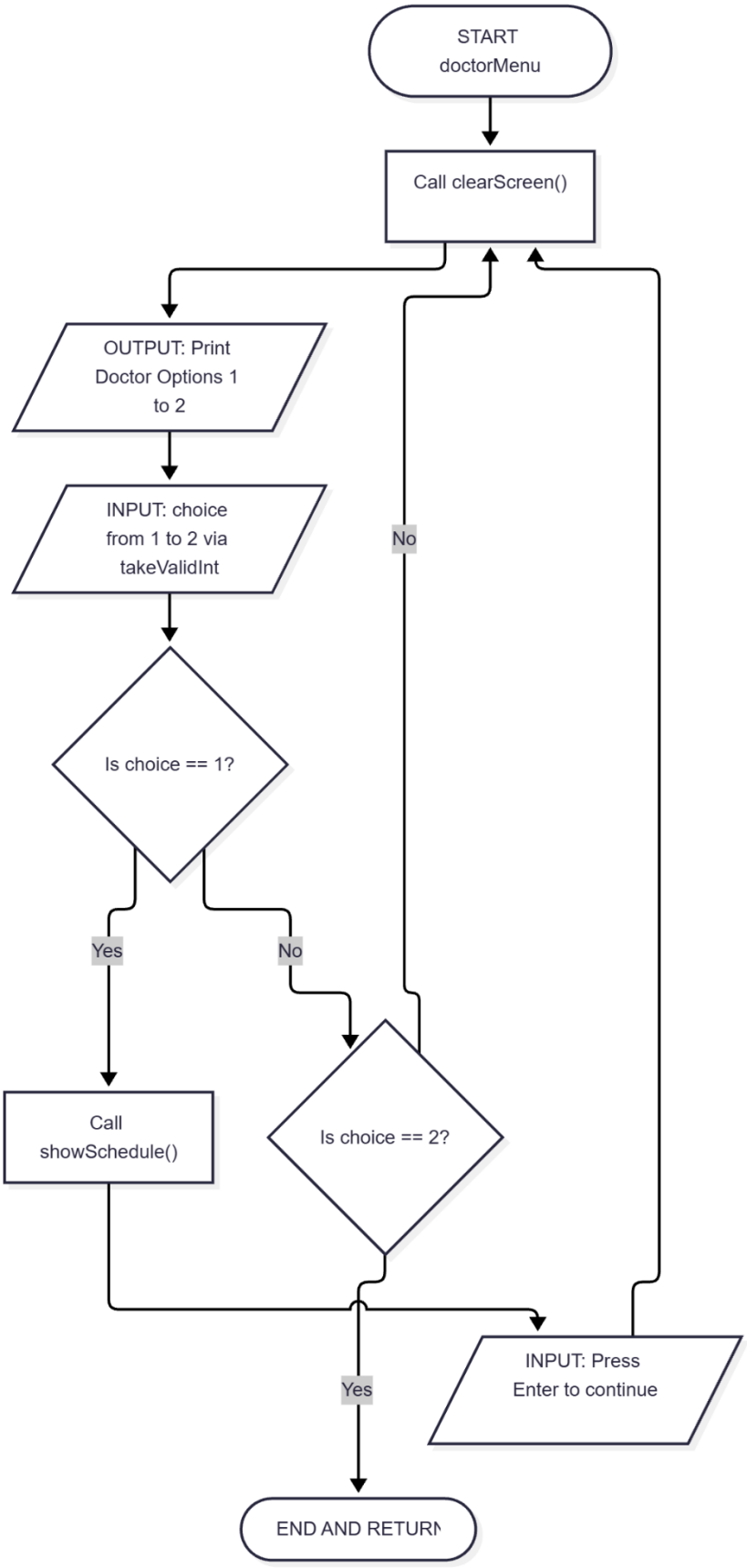
6. Flowchart

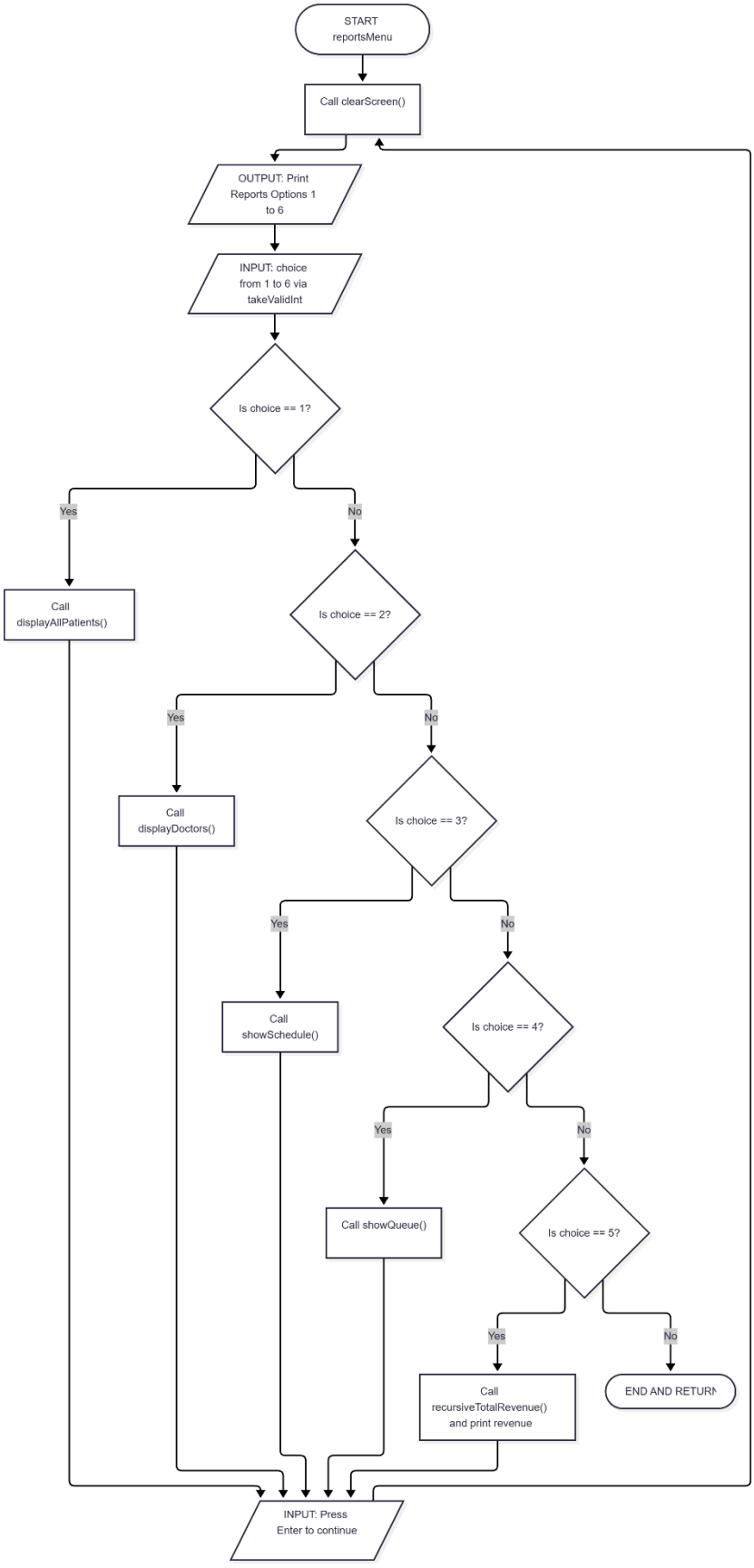

START Program
OUTPUT: 'WELCOME TO HOSPITAL MANAGEMENT SYSTEM'
Allocate dynamic memory for patients array
Is patients memory allocated?
No
Yes
OUTPUT: 'Memory allocation failed!'
END WITH CODE
Call loadHospitalData
Call mainMenu
Save Hospital Data to File
Free dynamically allocated patients memory
OUTPUT: 'Thank you for using system. Exiting...'
END WITH CODE (

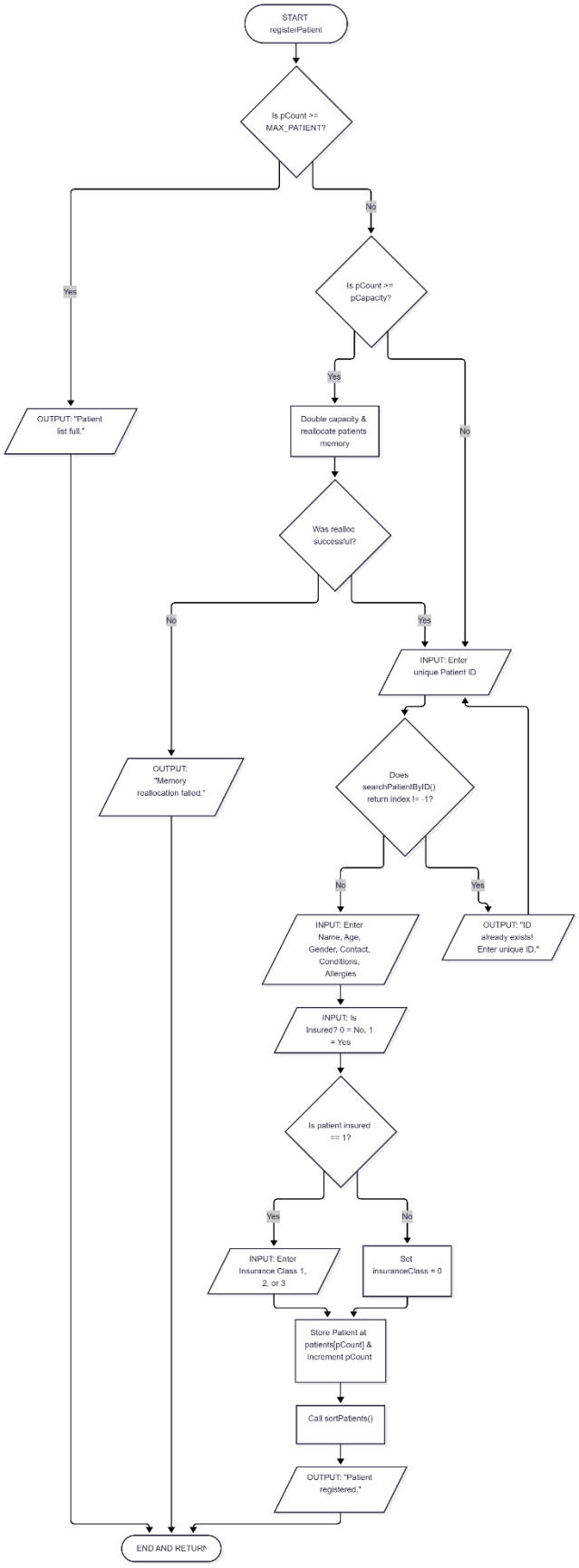

START mainMenu
OUTPUT: Print Main Menu Options 1 to 5
INPUT: choice from 1 to 5 via takeValidInt
Is choice == 1?
No
Yes
Call adminMenu
Is choice == 2?
No
Yes
Call receptionMenu
Is choice == 3?
No
Yes
Call doctorMenu
Is choice == 4?
No
Yes
Call reportsMenu
Is choice == 5?
No
Yes
END AND RETURN
Call clearScreen & saveHospitalData

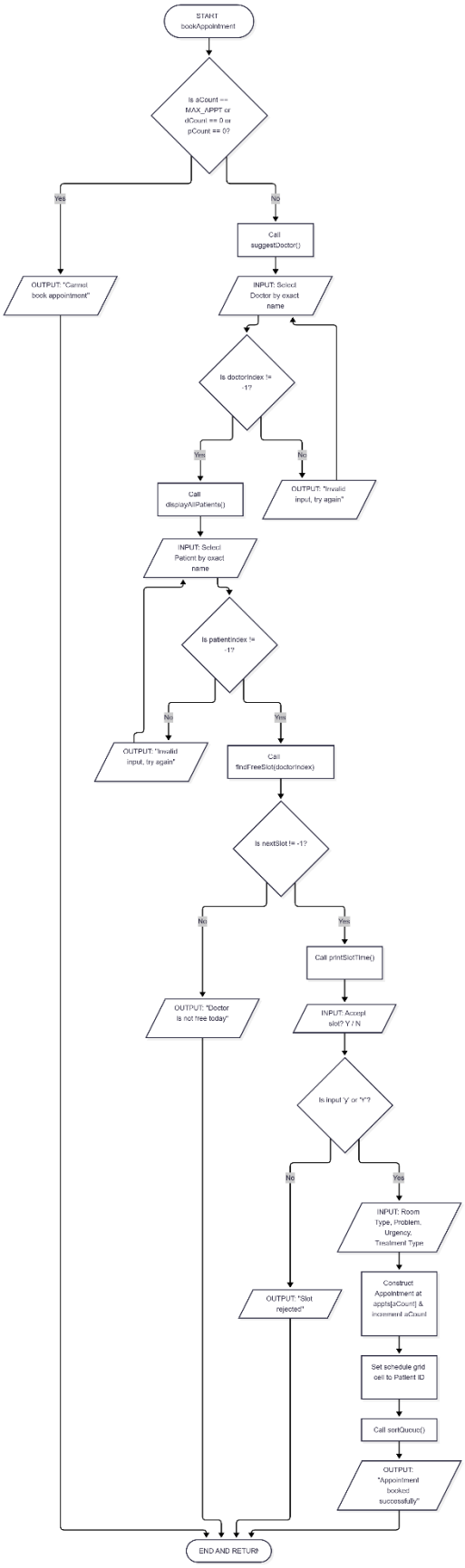

START
adminMenu
Call clearScreen()
OUTPUT: Print
Admin Options 1
to 3
INPUT: choice
from 1 to 3 via
takeValidInt
Is choice == 1?
No
Yes
No
Call addDoctor()
Is choice == 2?
Yes
No
Call
displayDoctors()
Is choice == 3?
INPUT: Press
Enter to continue
Yes
END AND RETURN

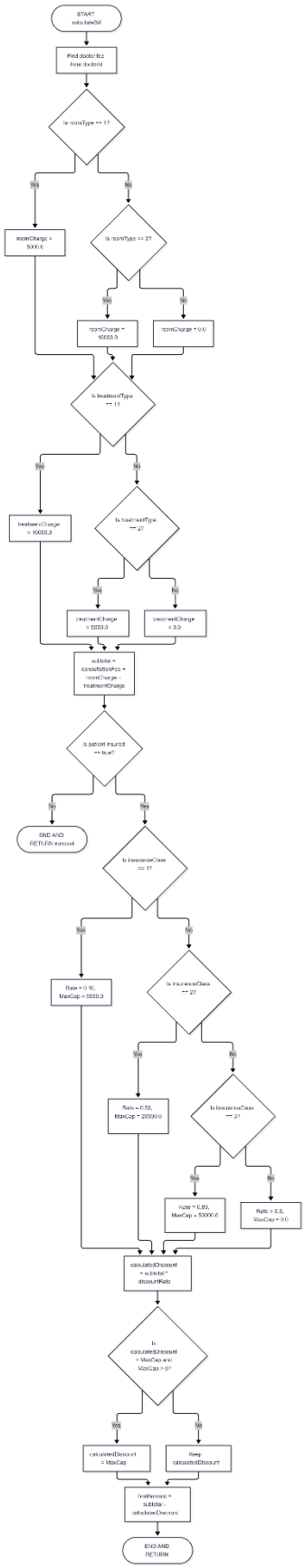

START receptionMenu
Call clearScreen()
OUTPUT: Print Reception Options 1 to 9
INPUT: choice from 1 to 9 via takeValidInt
Is choice == 1?
Yes
No
Call registerPatient()
Is choice == 2?
Yes
No
Call displayAllPatients()
Is choice == 3?
Yes
No
Call bookAppointment()
Is choice == 4?
Yes
No
Call cancelAppointment()
Is choice == 5?
Yes
No
Call completeAppointment()
Is choice == 6?
Yes
No
Call showQueue()
Is choice == 7?
Yes
No
Prompt Patient ID and call editPatient()
Is choice == 8?
Yes
No
Call payUnpaidBill()
END AND RETURN
INPUT: Press Enter to continue

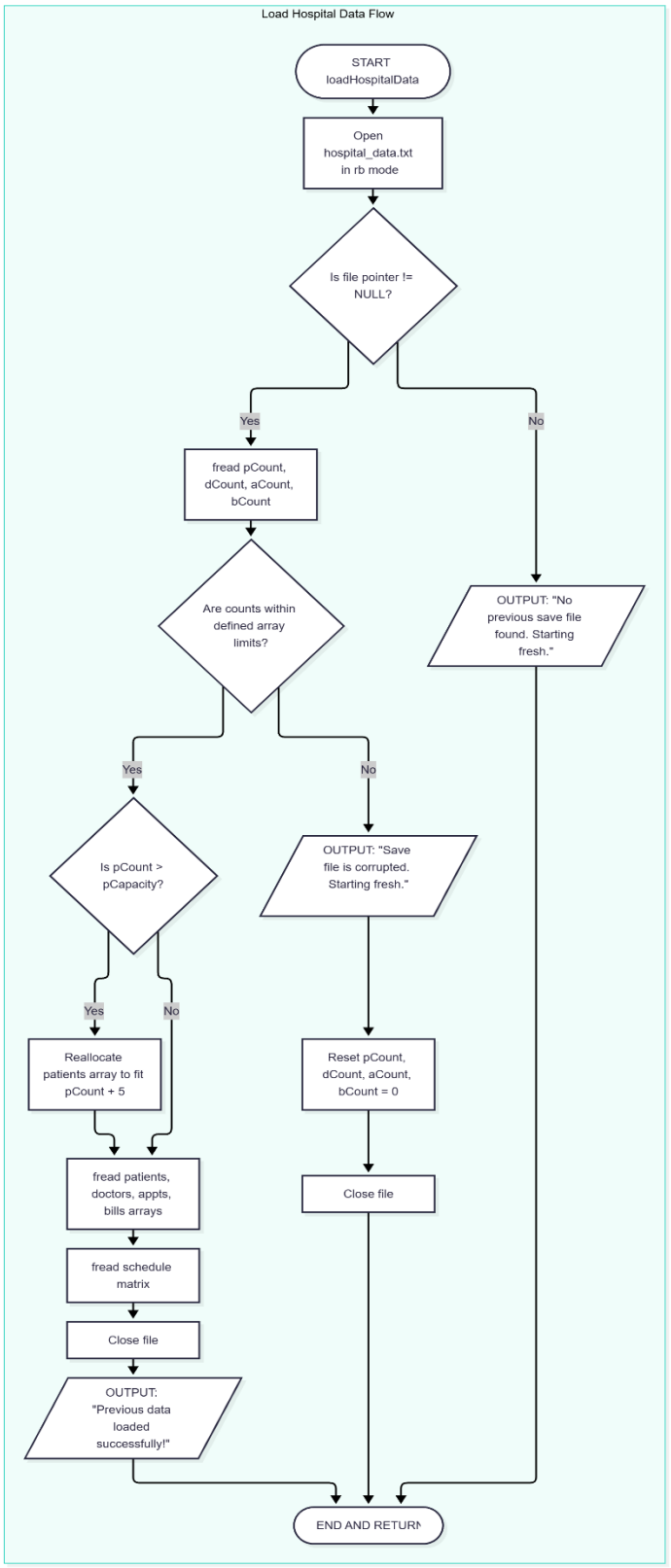

START
doctorMenu
Call clearScreen()
OUTPUT: Print
Doctor Options 1
to 2
INPUT: choice
from 1 to 2 via
takeValidInt
No
Is choice == 1?
Yes
No
Call
showSchedule()
Is choice == 2?
INPUT: Press
Enter to continue
Yes
END AND RETURN

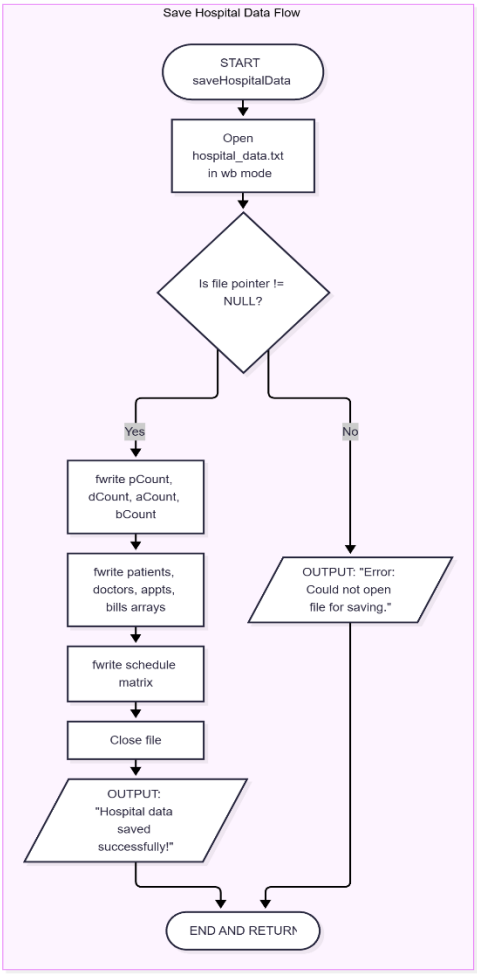

START
reportsMenu
Call clearScreen()
OUTPUT: Print
Reports Options 1
to 6
INPUT: choice
from 1 to 6 via
takeValidInt
Is choice == 1?
Yes
No
Call
displayAllPatients()
Is choice == 2?
Yes
No
Call
displayDoctors()
Is choice == 3?
Yes
No
Call
showSchedule()
Is choice == 4?
Yes
No
Call showQueue()
Is choice == 5?
Yes
No
Call
recursiveTotalRevenue()
and print revenue
END AND RETURN
INPUT: Press
Enter to continue


START registerPatient
Is pCount >= MAX_PATIENT?
Yes
No
OUTPUT: "Patient list full."
Is pCount >= pCapacity?
Yes
No
Double capacity & reallocate patients memory
Was realloc successful?
No
Yes
OUTPUT: "Memory reallocation failed."
INPUT: Enter unique Patient ID
Does searchPatientByID() return index != -1?
No
Yes
OUTPUT: "ID already exists! Enter unique ID."
INPUT: Enter Name, Age, Gender, Contact, Conditions, Allergies
INPUT: Is Insured? 0 = No, 1 = Yes
Is patient insured == 1?
Yes
No
INPUT: Enter Insurance Class 1, 2, or 3
Set insuranceClass = 0
Store Patient at patients[pCount] & increment pCount
Call sortPatients()
OUTPUT: "Patient registered."
END AND RETURN


START bookAppointment
Is aCount == MAX_APPT or dCount == 0 or pCount == 0?
Yes
No
OUTPUT: "Cannot book appointment"
Call suggestDoctor()
INPUT: Select Doctor by exact name
Is doctorIndex != -1?
Yes
No
Call displayAllPatients()
OUTPUT: "Invalid input, try again"
INPUT: Select Patient by exact name
Is patientIndex != -1?
No
Yes
OUTPUT: "Invalid input, try again"
Call findFreeSlot(doctorIndex)
Is nextSlot != -1?
No
Yes
OUTPUT: "Doctor is not free today"
Call printSlotTime()
INPUT: Accept slot? Y / N
Is input 'y' or 'Y'?
No
Yes
OUTPUT: "Slot rejected"
INPUT: Room Type, Problem, Urgency, Treatment Type
Construct Appointment at appts[aCount] & increment aCount
Set schedule grid cell to Patient ID
Call sortQueue()
OUTPUT: "Appointment booked successfully"
END AND RETURN


START calculateBill
Find doctor fee from doctorId
Is roomType == 1?
Yes
No
roomCharge = 5000.0
Is roomType == 2?
Yes
No
roomCharge = 10000.0
roomCharge = 0.0
Is treatmentType == 1?
Yes
No
treatmentCharge = 10000.0
Is treatmentType == 2?
Yes
No
treatmentCharge = 5000.0
treatmentCharge = 0.0
subtotal = consultationFee + roomCharge + treatmentCharge
Is patient insured == true?
No
Yes
END AND RETURN invoice
Is insuranceClass == 1?
Yes
No
Rate = 0.10, MaxCap = 5000.0
Is insuranceClass == 2?
Yes
No
Rate = 0.05, MaxCap = 20000.0
Is insuranceClass == 3?
Yes
No
Rate = 0.02, MaxCap = 50000.0
Rate = 0.0, MaxCap = 0.0
calculatedDiscount = subtotal * discountRate
Is calculatedDiscount > MaxCap and MaxCap > 0?
Yes
No
calculatedDiscount = MaxCap
Keep calculatedDiscount
finalAmount = subtotal - calculatedDiscount
END AND RETURN





Flowchart.pdf also attached

7. Testing

Test Cases

Test Case No.	Description	Input	Expected Output	Actual Output	Pass / Fail
TC-01	Load existing hospital data on program startup	Program launch	Attempting to load previous data... Previous data loaded successfully!	Attempting to load previous data... Previous data loaded successfully!	Pass
TC-02	Invalid choice selection in Reception Menu (Invalid Input)	10	Please enter a valid value between 1 and 9 (inclusive)	Please enter a valid value between 1 and 9 (inclusive)	Pass
TC-03	Attempting to add a doctor with an existing ID (Invalid Input)	1 (Option) Doctor ID: 1	ID already exists.	ID already exists.	Pass
TC-04	Booking appointment for a non-existent doctor specialization (Invalid Input)	Specialization: Pediatric	No doctor available for this specialization.	No doctor available for this specialization.	Pass
TC-05	View patient queue sorted by emergency status	4 (Reports) 4 (Queue)	Patient Usman Ahmed marked as [EMERGENCY] at top of queue	1. Usman Ahmed [EMERGENCY] - Time: 09:00	Pass
TC-06	View doctor daily schedule matrix	4 (Reports) 3 (Schedule)	Display grid showing FULL or AVBL slots per doctor	Grid displayed showing FULL for booked slots and AVBL for free slots	Pass
TC-07	Process bill payment and calculate change	Payment: 2000.00 for Amount: 1500.00	Payment successful. Change: 500.00	Payment successful. Change: 500.00	Pass

Test Case No.	Description	Input	Expected Output	Actual Output	Pass / Fail
TC-08	Recursive total revenue report calculation	4 (Reports) 5 (Revenue)	Total revenue is (including unpaid bills): 1500.00	Total revenue is (including unpaid bills): 1500.00	Pass
TC-09	Display all registered patients	4 (Reports) 1 (Patients)	Complete detailed list of registered patients	Complete detailed list of registered patients	Pass
TC-10	Save data and exit system	5 (Exit)	OPERATIONS COMPLETE...ATTEMPTING TO SAVE DATA Hospital data saved successfully!	OPERATIONS COMPLETE...ATTEMPTING TO SAVE DATA Hospital data saved successfully!	Pass

8. Screenshots

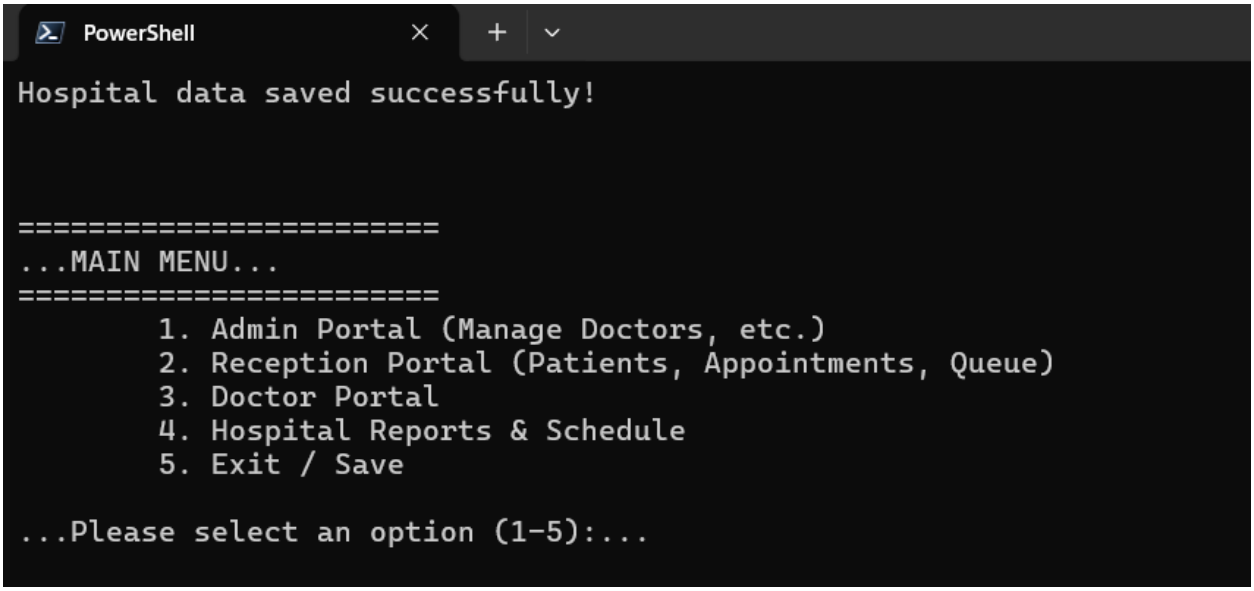


Figure 1: Main Menu

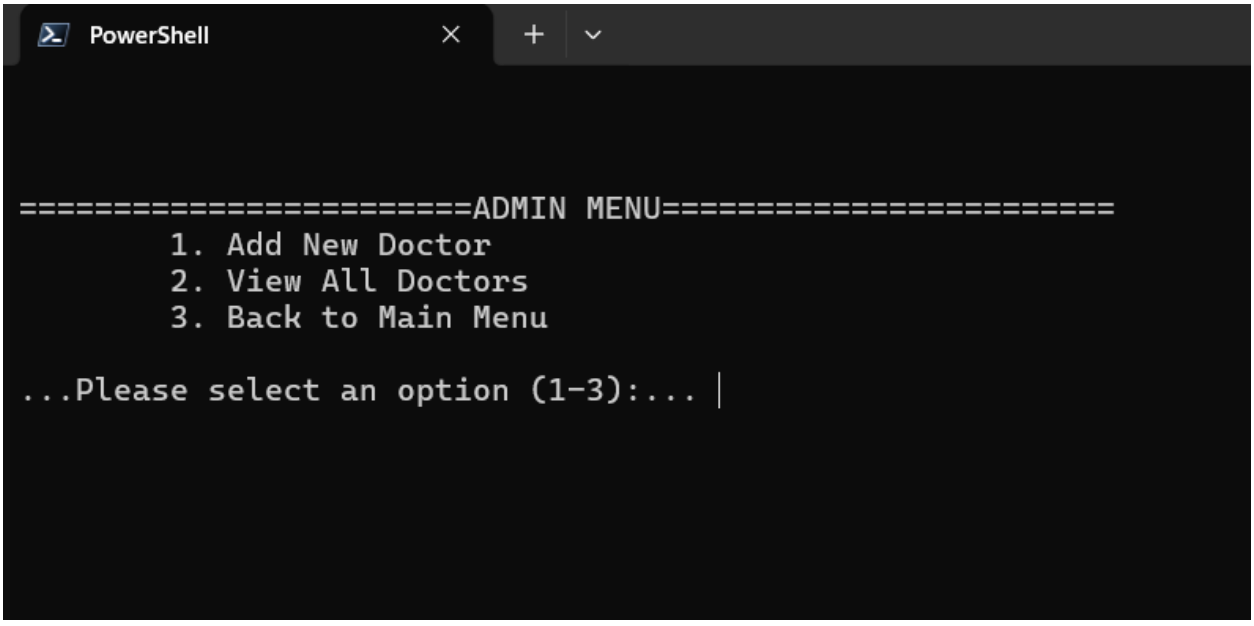


Figure 2: Admin Menu

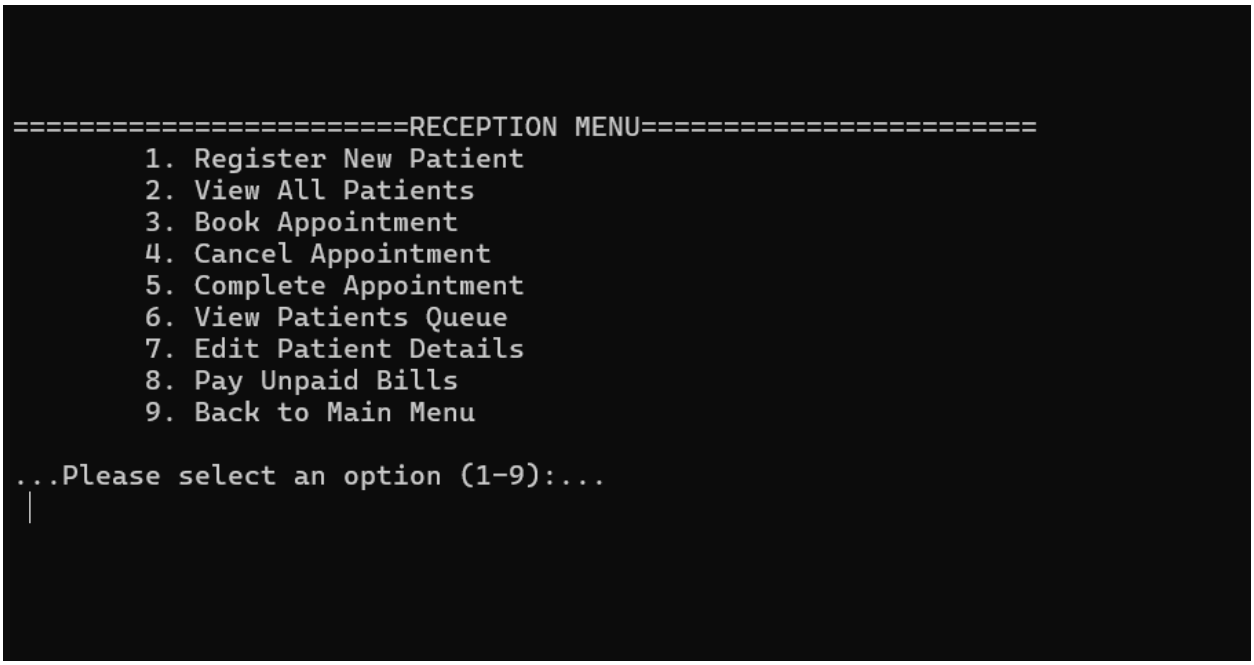


Figure 3: Reception Menu

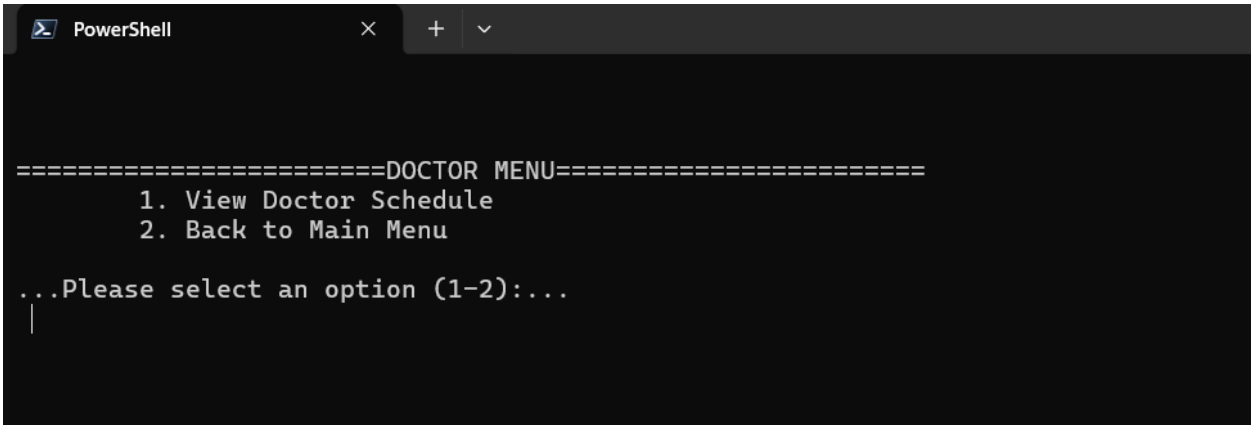


Figure 4: Doctor Menu

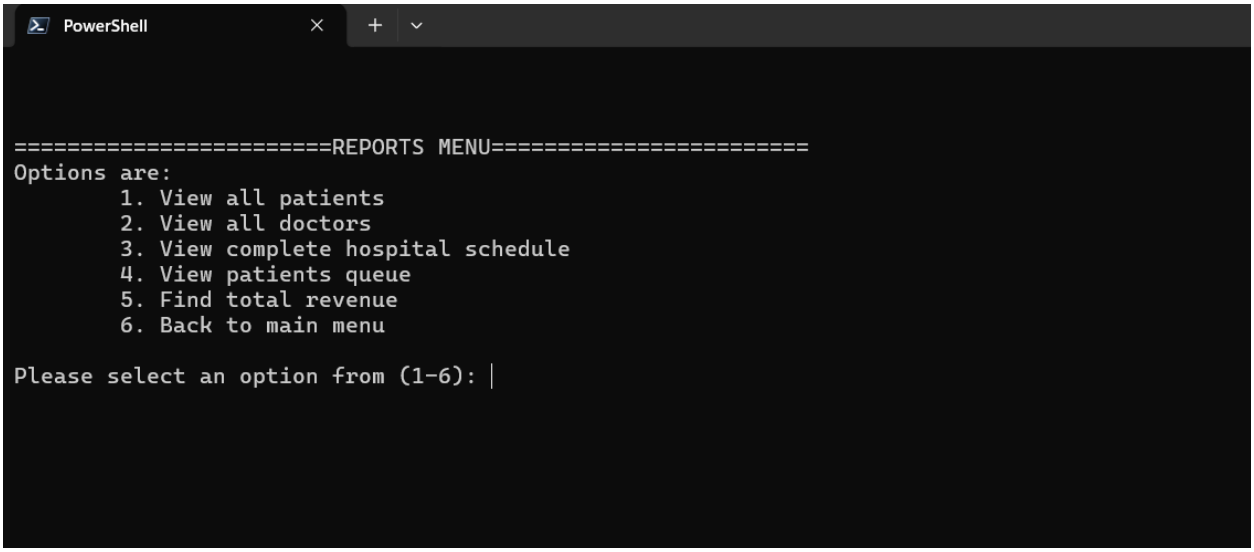


Figure 5: Reports Menu

Reports to show an example state of system

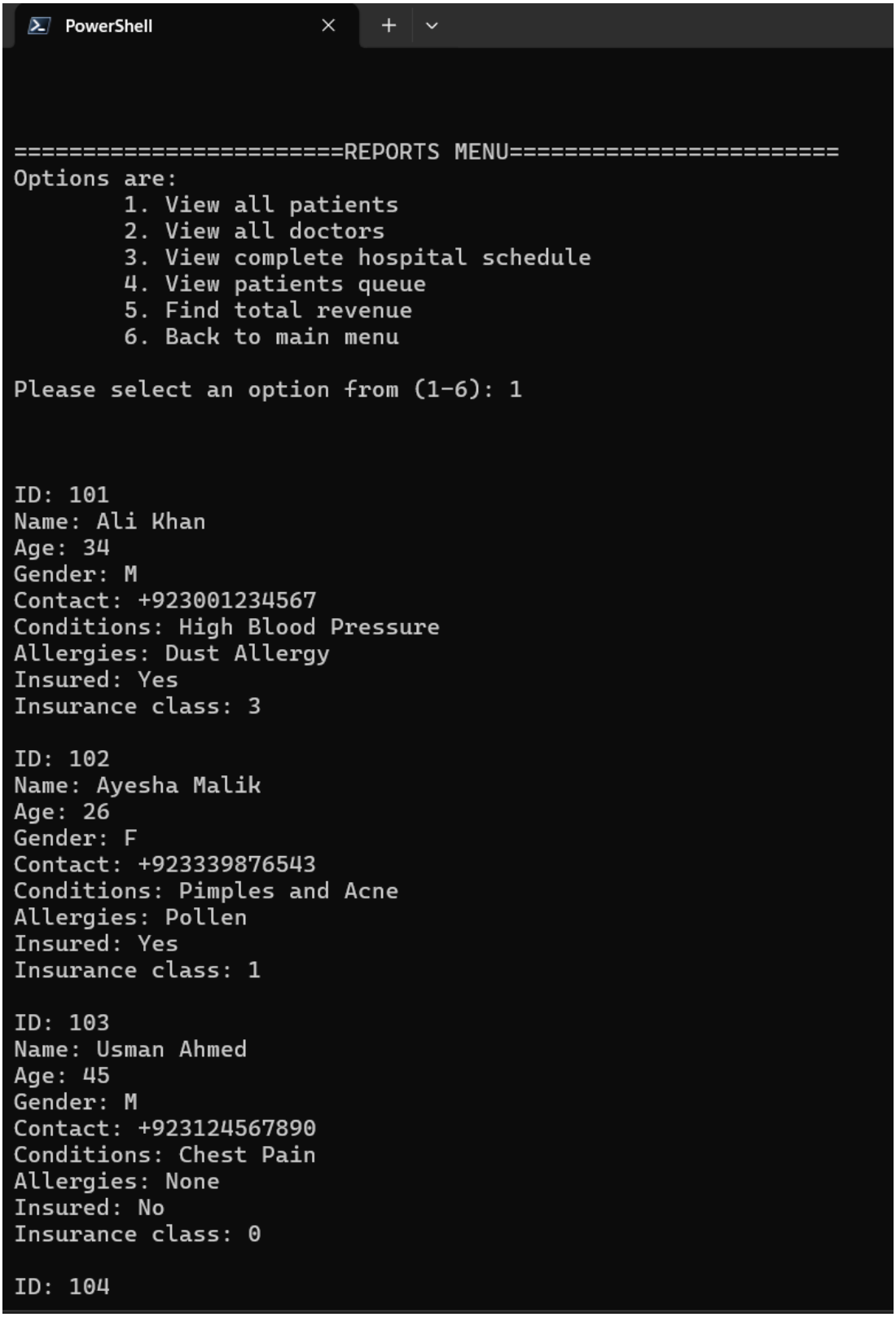


Figure 6: Stored Patients

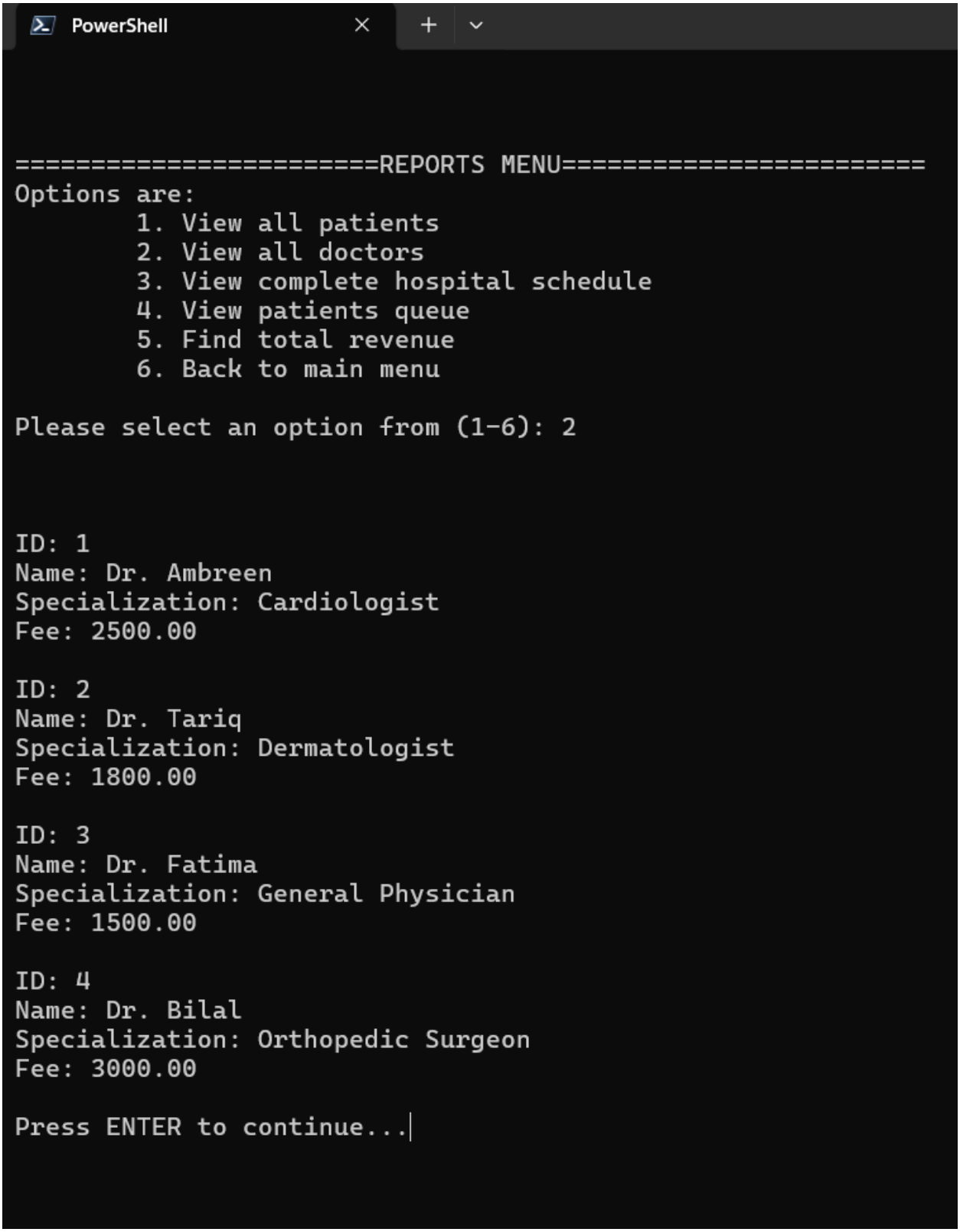


Figure 7: Stored Doctors

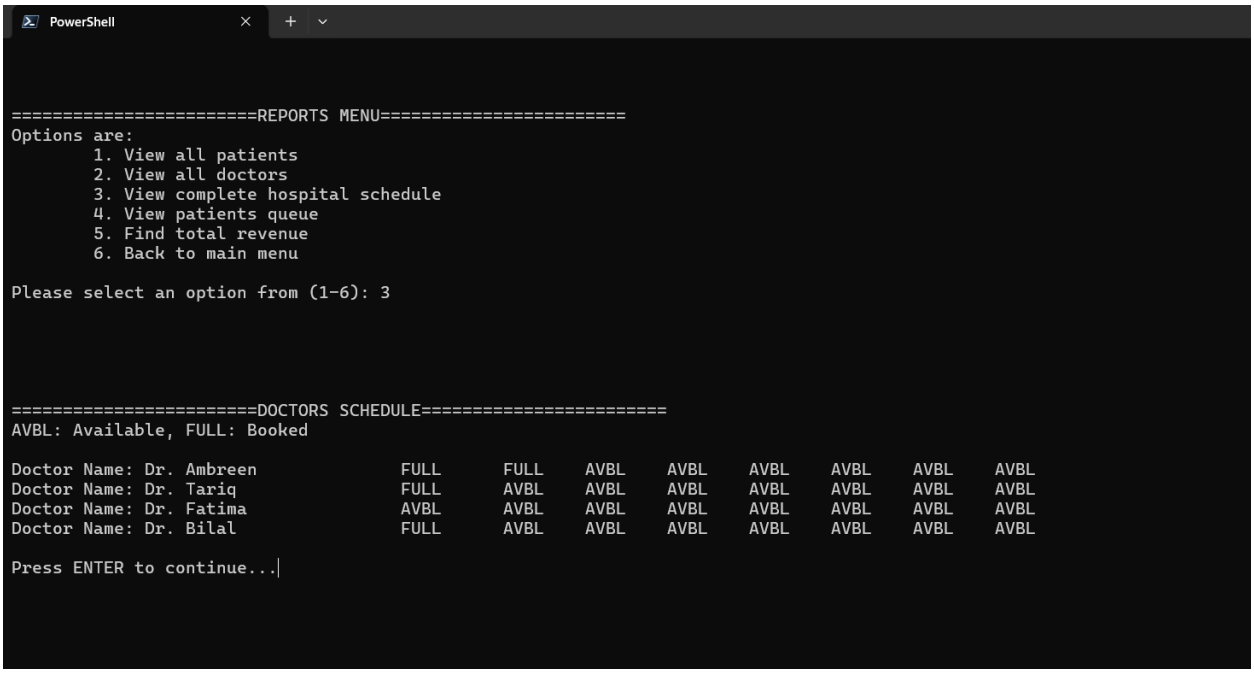


Figure 8: Current Schedule

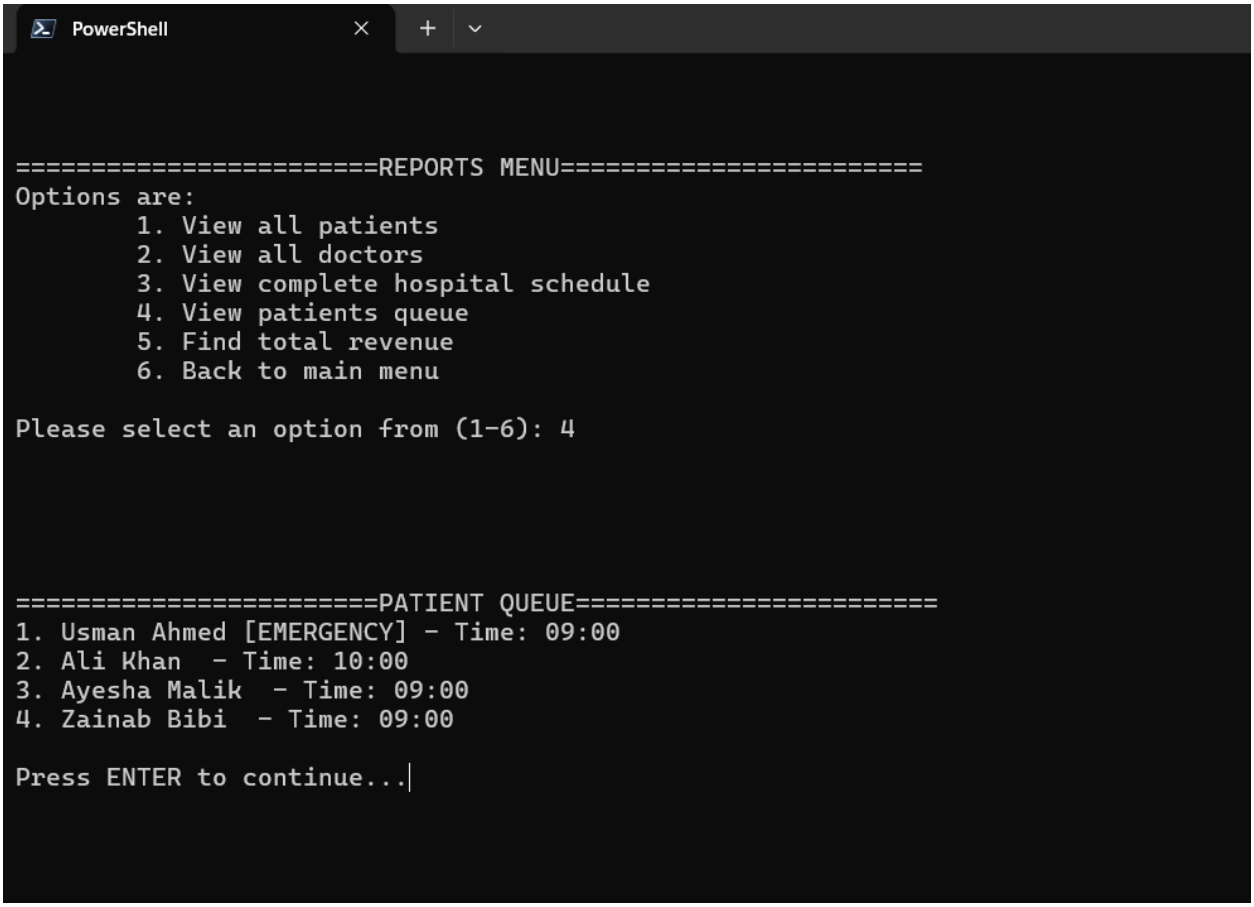


Figure 9: Patient Queue

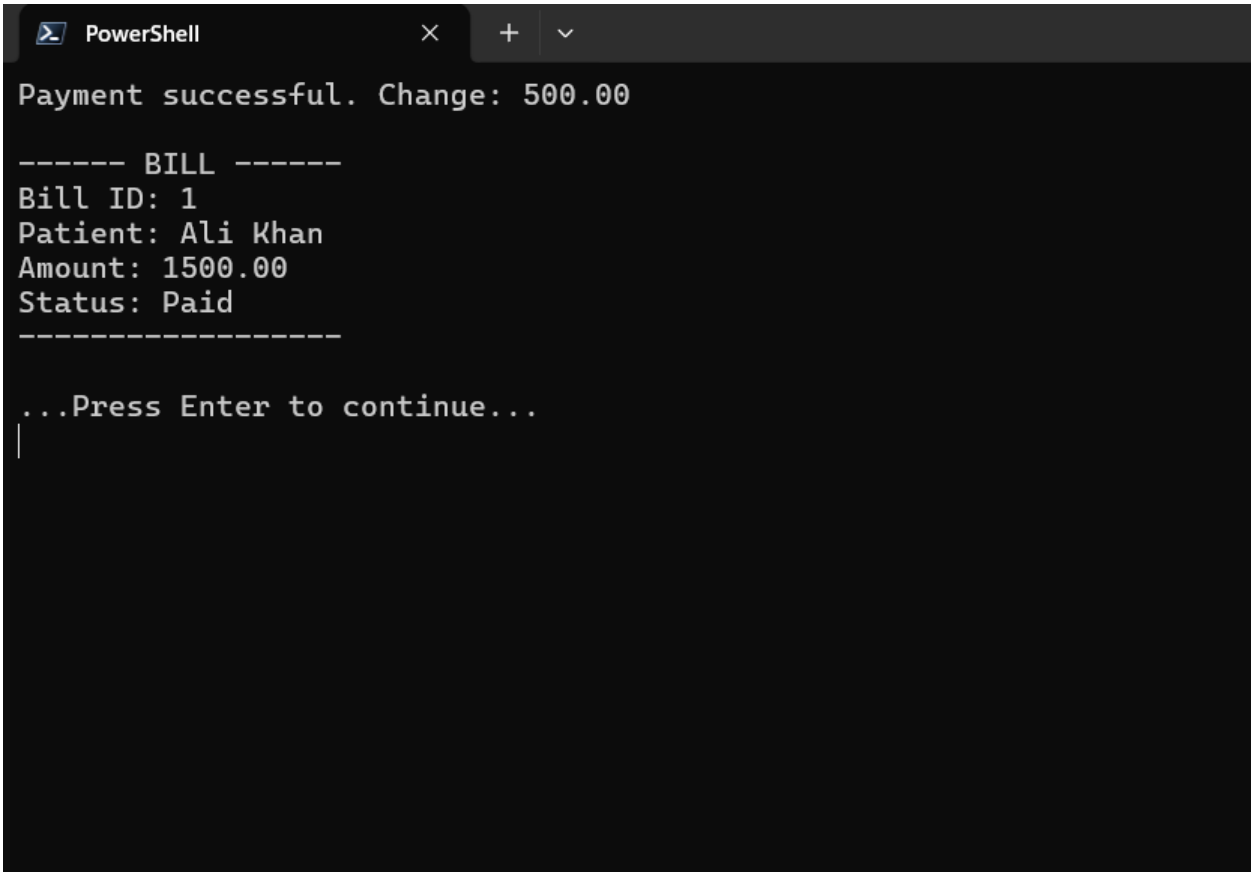


Figure 10: Bill Payment

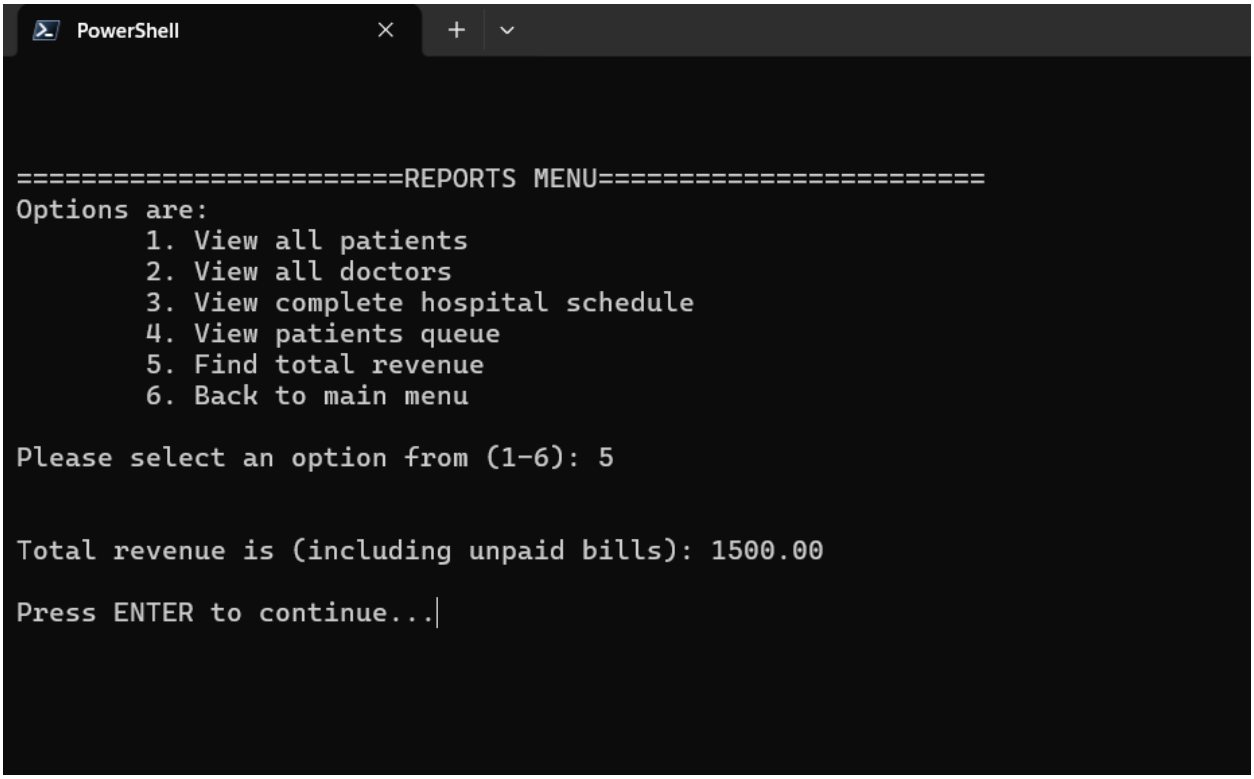


Figure 11: Hospital Revenue

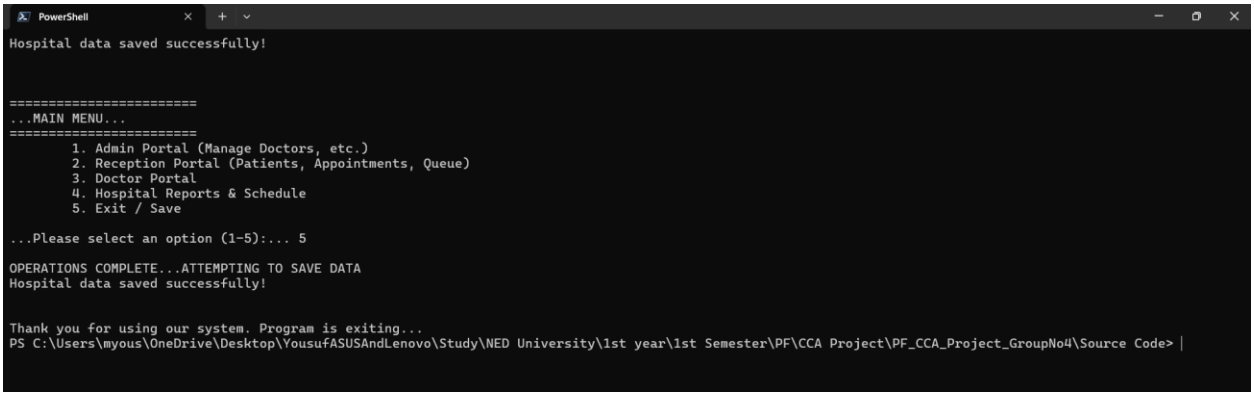


Figure 12: Demonstration of file saving

Erroneous Data Entry

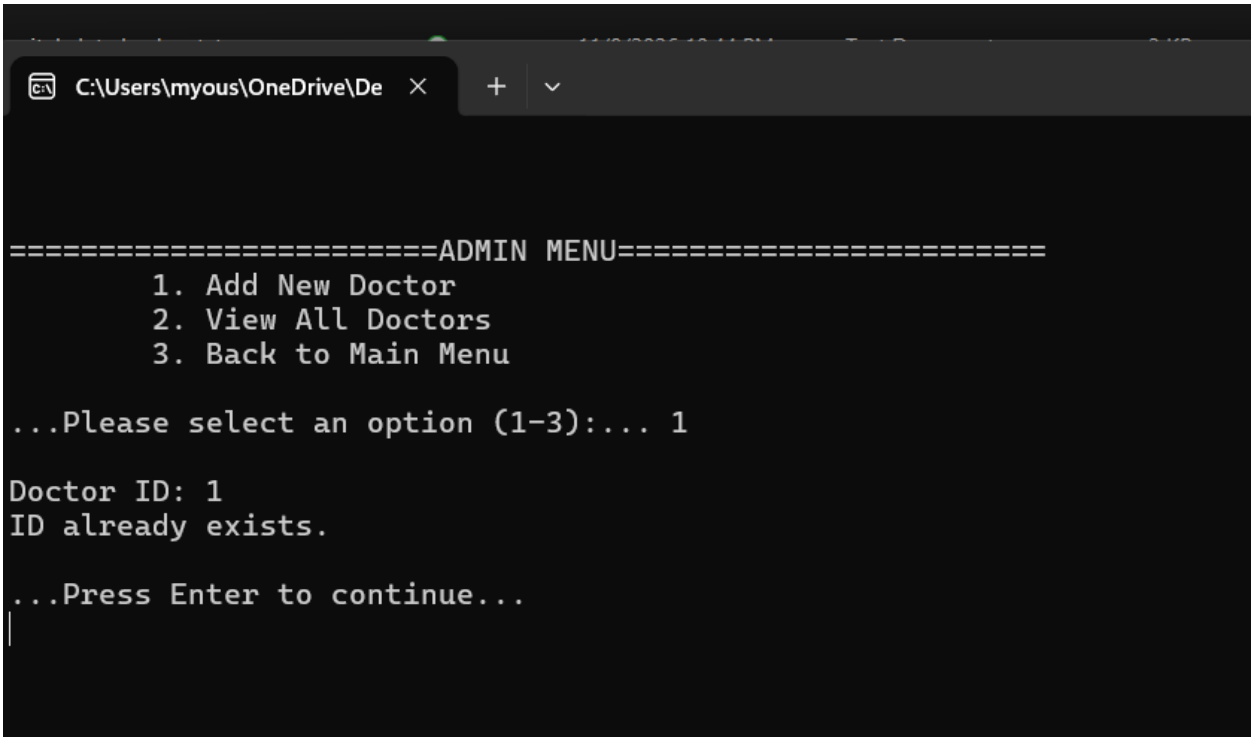


Figure 13: Demonstration of duplicate ID handling

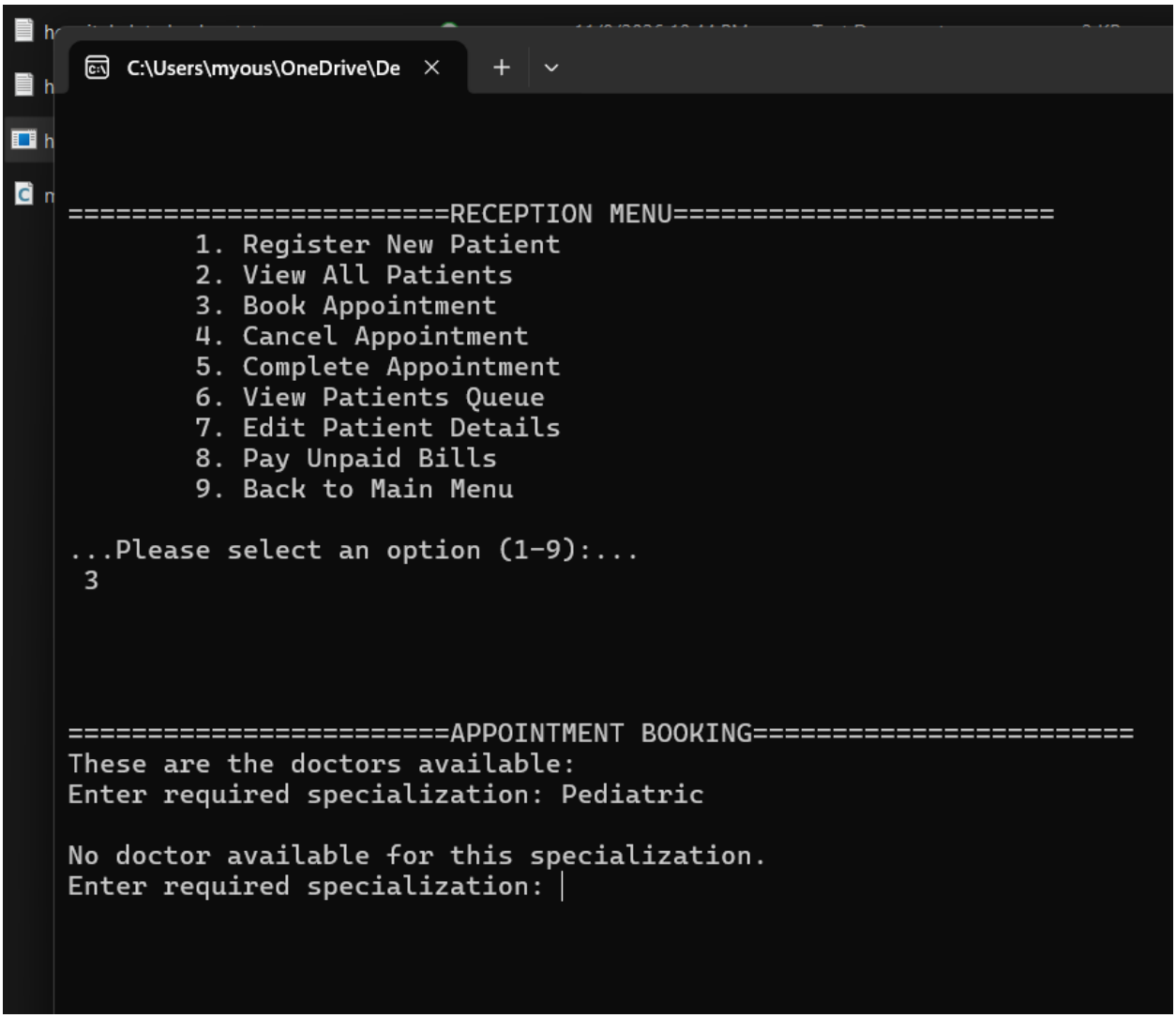


Figure 14: Demonstration of unavailable specialization

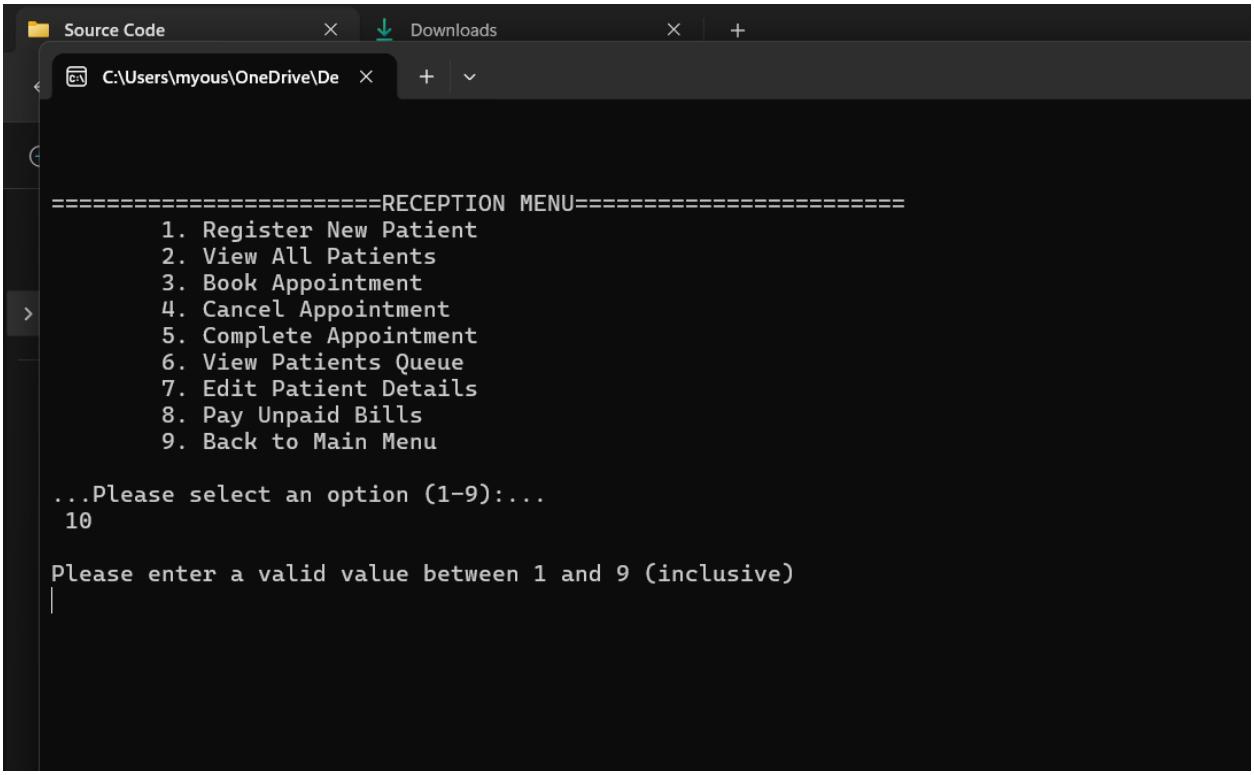


Figure 15: Demonstration of invalid option input screenshot

9. Conclusion

The Hospital Management System successfully does the main administrative tasks including patient registration, doctor schedule tracking, emergency priority queue, insurance-adjusted billing, and binary file persistence. Developing this system provided valuable experience in managing dynamic memory allocation, structured data types, modular function architecture, recursive algorithms, and file handling in C. For future enhancements, the application could be improved by transitioning from a console line interface to a graphical user interface (GUI) alongside implementing database integration (such as SQLite) for relational storage.

Checklist before converting to PDF

- ☑ Title page filled in with names, roll numbers and GitHub link
- ☑ All 9 sections completed
- ☑ Italic instruction lines deleted
- ☑ Screenshots are clear and captioned
- ☑ Saved as Project_Report.pdf